

NISRIA COMMAND CENTER

Sasa Bot: Architecture & Logic Tree

How a WhatsApp message becomes an action, every decision node, and exactly where to improve it.

Live at command.nisria.co · Built by SEV7EN Marketing AI

Prepared 2026-06-26 · Status: new architecture deployed, gated by maintenance

Source of truth: this document is traced from the live code (file:line cited per node).

1. What this is, in plain terms

Sasa is an office, not a single brain. A message arrives like a customer at a front desk. A receptionist routes it to the right clerk. Each clerk has only their own equipment. The clerk's job is to **understand** what is wanted; the system does the action and writes the reply. That separation is the whole design.

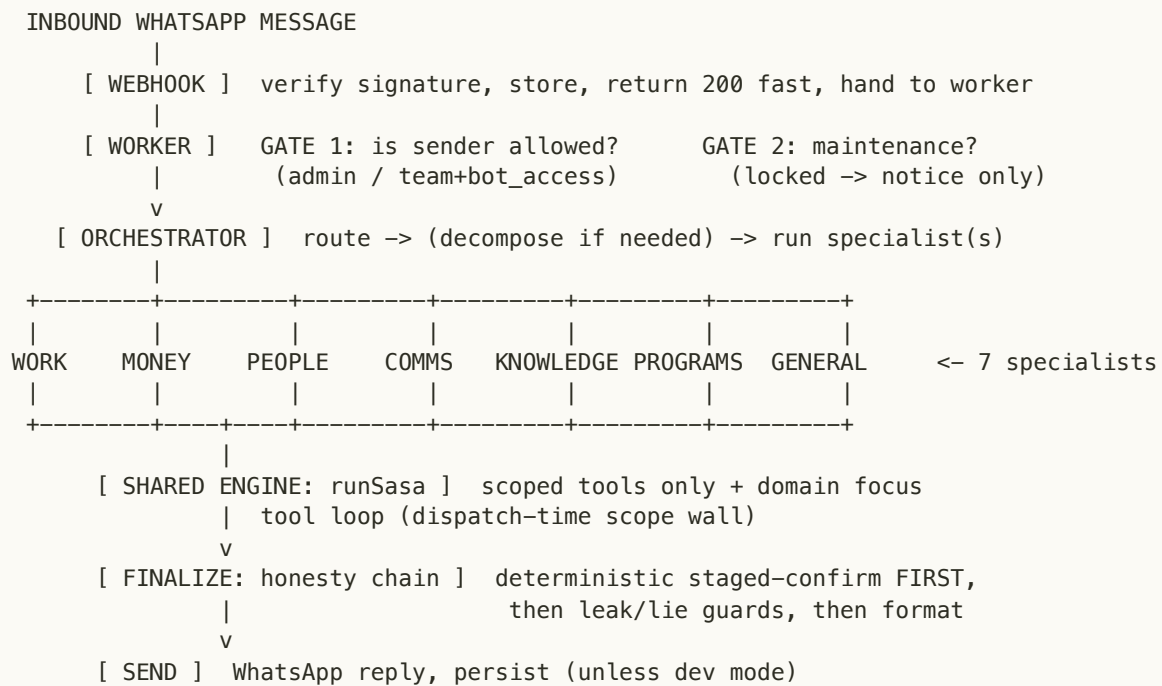
The bot has moved through three versions. **Mono**: one brain with every tool, no routing, easily confused. **Mesh** (live now): a router plus seven scoped specialists on one shared engine, with an honesty chain that refuses false claims. **Rails** (in progress): narrow the model to understanding only, and make the action and the reply deterministic, so a clear command produces one correct, gradable result.

The recurring failure was never routing or tools. It was the last step: letting the model write the "what happened" sentence freely, where it could say "Logged" for a payment that was only staged. The fix is to take that pen away. This document maps every node so each one can be improved on purpose, not by guesswork.

CURRENT STATE (HONEST)

- New architecture is **deployed** to prod and verified live (the new `/api/eval/replay` route answers on the apex; a real signed webhook drove the full pipeline).
- The bot is **gated by maintenance** right now: only the owner number is served; members get a single notice. Lifting it is one env flip plus redeploy.
- The brain runs on a funded Anthropic key set in prod. Swap to a dedicated bot key when convenient.

2. The architecture in one picture



Three things from the latest work sit at fixed nodes: the scope-leak guard at the specialist, the deterministic staged-confirmation and leak-strip in finalize, and the routing telemetry (`mesh.routed` / `mesh.completed`) at the orchestrator. They are called out per node below.

3. The runtime logic tree, node by node

Each node lists what it does, where it lives, its current weak point, and the concrete improvement. This is the work list.

1 Webhook (fast intake)

app/api/whatsapp/webhook/route.ts:61

Verifies Meta's HMAC signature, resolves the sender to a contact, stores the inbound message, enqueues a job, triggers the worker, and returns 200 within Meta's timeout. The slow brain runs separately so Meta never sees a timeout.

Weak point: a bad or missing `WHATSAPP_APP_SECRET` rejects everything; media arrives as a reference that must be fetched later.

Improve: add a dead-letter log for signature failures and malformed payloads so silent drops are visible, not invisible.

2 Gate 1 — is the sender allowed?

worker/route.ts:104

Allowed only if role is admin (owner/founder on the operators allowlist) or a team member with `bot_access=true`. Everyone else is logged as `whatsapp.ignored` and gets no reply. The 727 line is invite-only.

Weak point: a legitimate new team member with no `bot_access` flag is silently ignored, which looks like the bot is broken.

Improve: a one-line "you are not enrolled yet, ask Nur" reply for known-but-unflagged team numbers, so silence never reads as failure.

3 Gate 2 — maintenance

worker/route.ts:213

When `MAINTENANCE_MODE=1`, only the allowlisted owner number gets full service. Everyone else gets one canned maintenance notice (throttled to once per six hours) and nothing else runs: no brain, no tools, no writes. This gate is currently ON.

Weak point: the gate is all-or-nothing and lives in an env var, so toggling needs a redeploy; there is no partial / read-only mode.

Improve: a soft mode that lets reads through but blocks writes during a quality pass; move the toggle to a DB flag so it flips without a redeploy.

4 Coalesce the burst

worker/route.ts:396

A durable per-contact claim merges several quick messages ("salary 15k", "+5k transport", "for Mark") into one turn so the bot answers the whole thought, not each fragment. Losers defer to the winner.

Weak point: the coalesce window is a fixed wait; too short splits a thought, too long feels slow.

Improve: end the window early on a sentence-final signal (a question mark, "thanks", a long pause) instead of always waiting the full timer.

5 Window-open flush

worker/route.ts:431

When a contact messages in, any reply that was held earlier (because WhatsApp's 24-hour window was closed) can now be delivered. Gated by `isLive` so test and maintenance traffic never trigger a real send.

Improve: surface held-message count to the operator so nothing waits silently for days.

6 Orchestrator — route & (maybe) decompose

lib/agents/orchestrator.ts:69

Media goes through an intake pipeline to a domain. Text goes to `routeMessage` (regex first, Haiku model as fallback), returning a domain and a confidence. Low-confidence or multi-intent messages are split into up to three steps, each run as its own specialist. There is no monolith fallback by design.

Weak point: routing is score-by-keyword-length; "remind me to send X" can tip to comms instead of work. Routing was ~86% on the hard set.

Improve (the big lever): this is where the Rails plan lands. Replace free tool-choice downstream with a structured parse (intent + slots), so the path becomes route to deterministic action. See section 7. Emits `mesh.routed` for measurement.

7 Specialist — scope the tools, focus the lane

lib/agents/specialists/index.ts:54

Looks up the domain's tool list, fails closed if the scope is somehow empty (never runs unscoped), injects the domain focus plus the `no-scope-leak` rule, and calls the shared engine. This is the security wall: a money turn physically cannot reach a people tool.

Weak point: the focus is prose the model can still drift around; tool choice still happens inside the engine, not here.

Improve: move tool choice out of the engine and into a deterministic intent to tool table here, per the Rails plan.

8 Shared engine — the tool loop

lib/agents/sasa.ts (runSasa)

Builds the system prompt (persona, Brain grounding, the real current date via the clock), then loops: the model proposes a tool, a **dispatch-time scope check** rejects any tool outside this domain, the tool runs, the result is fed back. Staging tools (payment, merge, delete) return "Ready to ... reply yes" with a `staged` flag.

Weak point: the model can still call the wrong in-scope tool, or loop asking instead of acting.

Improve: with the parse layer in place, the engine's tool loop shrinks to "execute the parsed action," removing most wrong-tool turns.

9 Finalize — the honesty chain

lib/agents/sasa.ts (finalize)

The reply passes through ordered guards. **PRIMARY (new): deterministic staged-confirmation** — if any tool staged something, the reply IS the tool's own line, so the model can never say "Logged" for a staged payment. Then: send-state truth, fake-staging, completed-but-staged, plural-send, fabricated-number, the **meta/scope-leak strip**, apology cap, and a completion-mismatch backstop. Finally WhatsApp formatting and chunking.

Weak point: the later guards are catch-after-the-fact; each new lie shape needs a new guard (the game we are leaving).

Improve: as Rails extends, every action's confirmation comes from the tool result, so the guards become a near-silent backstop. A guard firing should be an alert, not normal.

10 Orchestrator finalize-with-guard

lib/agents/orchestrator.ts:149

A last cross-domain leakage check on the real tools that ran, then emits `mesh.completed` with the tools and step count for measurement.

Improve: include the parsed intent and confidence in the telemetry so every turn is gradable against the golden set automatically.

11 Send

lib/whatsapp.ts (sendTextAndLog)

Sends the reply over WhatsApp and persists it, except in dev mode (owner/harness), which reroutes and skips persistence with a `[DEV]` prefix.

Improve: confirm delivery via Meta's status callback and retry or flag on failure, so a dropped send is never assumed delivered.

4. The seven specialists and their tools

Each specialist is the shared engine scoped to one domain's tools. A specialist can never touch another domain's tools. Percentages are share of real transcript usage.

DOMAIN	MODEL	TOOLS (SCOPED)
work tasks, calendar	Haiku	create_task, complete_task, reopen_task, update_task, delete_task, add_task_comment, list_task_comments, link_task_dependency, list_task_dependencies, query_calendar, check_conflicts, create_event, move_event, delete_event, complete_calendar_event, list_tasks, read_brief, member_activity, dispatch_meeting_bot
money payments, donations (admin only)	Sonnet	record_payment, update_payment, delete_payment, schedule_payment, mark_payment_paid, query_donations, lookup_donor, newest_donor, finance_summary, latest_gift, list_campaigns, list_payroll, list_bank_transactions, donor_activity, log_payout, add_donor, update_donor, add_campaign, update_campaign, set_monthly_goal, log_team_payment, set_beneficiary_funding
people team, contacts, beneficiaries	Sonnet	team_detail, list_team, add_team_member, update_team_member, find_beneficiary, list_beneficiaries, add_beneficiary, update_beneficiary, delete_beneficiary, merge_beneficiary, approve_case, decline_case, move_case, edit_case, merge_case, delete_case, add_contact, update_contact, activate_member, set_bot_access, import_contacts, merge_contact, delete_contact, set_public_profile
comms outbound messaging (admin only)	Sonnet	message_person, post_to_group, send_file_to_person, relay_to_colleague, draft_email, draft_thank_you, draft_all_thank_yous, draft_post, flag_to_nur, run_group_digest, show_outbound_audit, read_contact_thread, read_email, search_inbox, inbox_status, mark_handled, post_to_social, publish_social_post, send_newsletter, show_draft, list_groups, group_activity, list_content
knowledge documents, Brain, grants	Haiku	search_documents, read_document, summarize_document, file_document, delete_document, list_learned, edit_brain_section, query_memory, list_grants, prepare_grants, refresh_grants, add_grant, update_grant_status, pursue_opportunity, find_studio_doc, transfer_drive_file, list_assets
programs Maisha inventory, wishlist	Haiku	list_inventory, add_inventory_item, update_inventory_item, list_wishlist, add_wishlist_item, update_wishlist_item, fund_wishlist_item
general greetings, ambiguous	Haiku	lookup_contact, search_history, remember_fact, flag_for_clarity, agent_activity

Cross-cutting tools available to every specialist: *lookup_contact, search_history, remember_fact, flag_for_clarity, agent_activity*. Permission: *money and comms are admin-only; the rest serve both admin and enrolled team (with the PII wall on people)*.

5. The honesty chain in detail (finalize)

Guards run in order. The first that matches rewrites the reply and the rest stand down. The top one is the new deterministic path; the others are the historical catch-after guards that Rails will make near-silent.

GUARD	WHAT IT CATCHES
deterministicStagedConfirm new	Any staged action (payment, merge, delete). The reply becomes the tool's own honest "Ready to ... reply yes" line, so the model cannot narrate a staged action as done.
send-state truth	"Did you message X?" answered from the real outbound log, not the model's guess.
fake-staging	"Reply yes to confirm" text when no staging tool actually ran.
completed-but-staged new	Backstop: "Logged" when only a staging tool ran. Now superseded by the primary path, kept as a safety net.
plural-send mismatch	"Both messages sent" when only one actually went.
fabricated-number	A KES/USD figure not in the user's words or any tool result.
meta/scope-leak strip new	"I'm scoped to comms tools only", "the rules I run on" and similar internal leaks.
apology cap	Repeated "I'm so sorry, Nur" condolence openers on routine turns.
completion-mismatch backstop	Any "done/created/sent" claim with no category-matched tool success this turn.

6. Known failure modes (from the real benchmark)

Measured on 89 cases where the command was clear and the old bot failed. The honest read: the dangerous money lies are now closed at the source; the remaining failures are wrong tool-choice and need the parse layer.

FAILURE	STATUS	FIX PATH
Payment shown as "Logged" when only staged	closed	Deterministic staged-confirm (node 9)
Case merge replied with money-shaped text	closed	Domain-neutral line + staged-confirm
"I'm scoped to comms tools only this turn"	closed	Leak strip + route "remind me" to work
Meta-narrative ("the rules I run on now")	closed	Leak strip
Created a task when asked to edit one	open	Intent parse (wrong-tool class)
Calendar "all done" with an unrequested delete	open	Intent parse + templated confirm
Invented a task list / a one-assignee limit	open	Parse + reply only from tool result
Asked a question the user already answered	open	Parse fills slots from the message
Contradicted itself on a task lookup	open	Deterministic read + ask-once

Note on grading: 13 of the original "failures" were a judge artifact (it wanted a confirm-step on task creation that Nur likely does not want), not real bot bugs. Counting those as correct puts the new bot near three-quarters on the hardest set before the parse layer.

7. How we improve it (the Rails plan)

One change, applied per domain: narrow the model to understanding, make the action and the reply deterministic.

TODAY (mesh)	TARGET (rails)
route -> specialist:	route -> specialist:
LLM picks tool + writes reply	1. LLM PARSE -> {intent, slots, confidence}
(wrong-tool + wrong-words bugs)	2. deterministic intent -> tool (scoped)
	3. execute, capture result
	4. templated confirmation from the result
	5. fallback to free chat only when
	intent = conversation / low confidence

Why this reaches near-100% on clear commands

- The model does one job it is good at: turn messy, multilingual phrasing into a structured intent and slots.
- Tool choice becomes a lookup table over the specialist's already-scoped tools, so wrong-tool turns disappear.
- The confirmation is rendered from what the tool actually did, so the bot cannot lie about an outcome.
- Every recognized action has one correct result, so the bot becomes **perfectly gradable** against a golden set.

Sequence

- **Done:** staged-action confirmations (money, merge, delete) deterministic; leak and routing fixes.
- **Next:** build the parse layer for **work** first (largest cluster), then money, people, comms, knowledge, programs.
- Each domain: intent catalog, intent to tool table, slot validators, confirmation templates, walls.
- **Measure once at the end** against the golden set on a safe sandbox deploy (sends dead, never prod), to respect key spend.

References in the repo: spec 005 (specs/005-deterministic-rails), ADR 0015 (docs/decisions/0015-intent-parse-then-execute.md), knowledge-tree nodes 406 to 408.

8. How to operate it

- **Unlock:** set `MAINTENANCE_MODE=0` in prod env, redeploy. **Lock:** set it to `1`, redeploy. The allowlist (owner number) always gets through.
- **Brain key:** `ANTHROPIC_API_KEY` in prod (type sensitive). It must be a funded key or the bot goes silent. Verify with a live brain test before unlocking.
- **Deploy oracle:** never trust the push. Confirm `command.nisria.co` is aliased to the new deployment and a discriminator route answers, then verify the brain with one real signed message.
- **Tests:** `npm run walls` (80 walls) plus `tsc` must be green; `predeploy.sh` gates the prod push.

This document is generated from the live code and is meant to be marked up. Pick any node, write the improvement beside it, and it becomes the next work item.